

# Implementation of Expectation-Maximization (EM) Algorithm in Python

## Aim

To implement the **Expectation-Maximization (EM)** algorithm in Python for clustering data using a **Gaussian Mixture Model (GMM)**.

## Apparatus Required

- Google Colab / Jupyter Notebook
- Python 3
- Libraries:
  - NumPy
  - Matplotlib
  - Scikit-learn (sklearn)

## Theory

The **Expectation-Maximization (EM)** algorithm is an iterative machine learning algorithm used to estimate the parameters of statistical models with hidden (latent) variables. It is commonly implemented using a **Gaussian Mixture Model (GMM)** for clustering.

The EM algorithm consists of two main steps:

- **Expectation (E-Step):** Calculates the probability that each data point belongs to each cluster.
- **Maximization (M-Step):** Updates the cluster parameters (mean, covariance, and weights) based on these probabilities.

The algorithm repeats these steps until convergence.

## Advantages:

- Can model overlapping clusters.
- Provides probabilistic cluster assignments.
- Suitable for complex data distributions.

## Disadvantages:

- Sensitive to initialization.
- May converge to a local optimum.

### Algorithm

1. Import the required libraries.
2. Generate or load a dataset.
3. Create a Gaussian Mixture Model with the desired number of clusters.
4. Fit the model using the EM algorithm.
5. Predict the cluster labels.
6. Plot the clustered data.
7. Display the cluster centers.

### Code (Google Colab)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

# Generate sample dataset
X, y = make_blobs(n_samples=300,
                  centers=3,
                  cluster_std=1.0,
                  random_state=42)

# Create EM model using Gaussian Mixture
gmm = GaussianMixture(n_components=3, random_state=42)
```

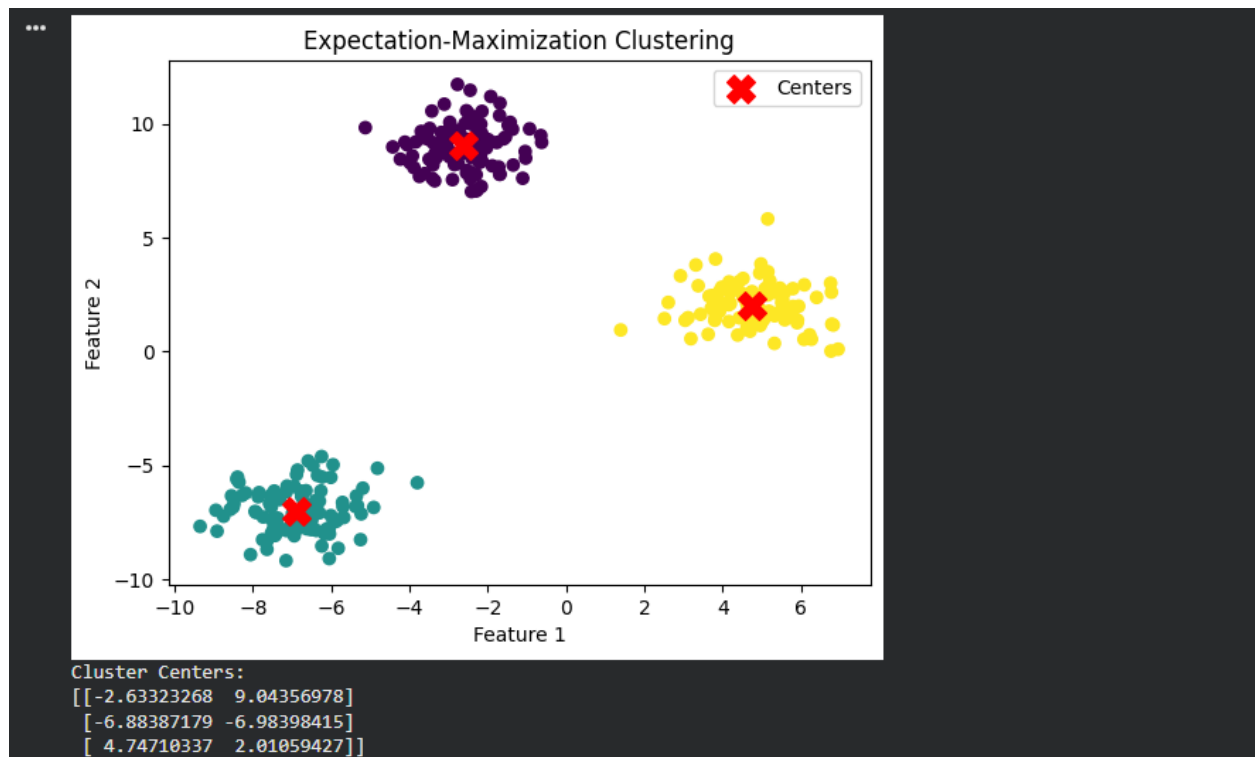
```
# Train model
gmm.fit(X)

# Predict clusters
labels = gmm.predict(X)

# Plot clusters
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
plt.scatter(gmm.means_[:,0], gmm.means_[:,1],
            c='red', marker='X', s=200, label='Centers')
plt.title("Expectation-Maximization Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()

# Display cluster centers
print("Cluster Centers:")
print(gmm.means_)
```

## **Sample Output**



## Result

The **Expectation-Maximization (EM)** algorithm was successfully implemented in Python using the **Gaussian Mixture Model (GMM)**. The algorithm effectively clustered the data into different groups, and the cluster centers were identified and visualized.